

Project report on
Smart_Expense_Tracker_AI_Financial_Advisor



Undertaken at:

Sopra Steria

24-06-2026 – 04-09-2026

Submitted by:
Gitesh Sharma
BTECH CSE 4th YEAR
Lloyd Institute Greater Noida
<https://github.com/realgitesh>

Under the guidance of:
Hasnain Zaheer
Technical Lead
Sopra Steria Group
Noida

TABLE OF CONTENTS

Internship Project Report

No.	Section	Page
1	Introduction	3–4
2	Purpose of the Project	5
3	Scope of the Project	6
4	Project Overview and Objectives	7
5	System Architecture	8–9
6	Application Modules and Functional Areas	10–12
7	Budget Monitoring and Financial Analytics	13
8	Reports and Data Presentation	14
9	AI Financial Advisor	15
10	Technology Stack and Development Environment	16
11	Security, Validation and Data Protection	17
12	Application of the Project	18
13	Testing and Current Project Status	19
14	Limitations and Future Scope	20
15	Conclusion	20
16	References	21
17	Bibliography	22

1. Introduction

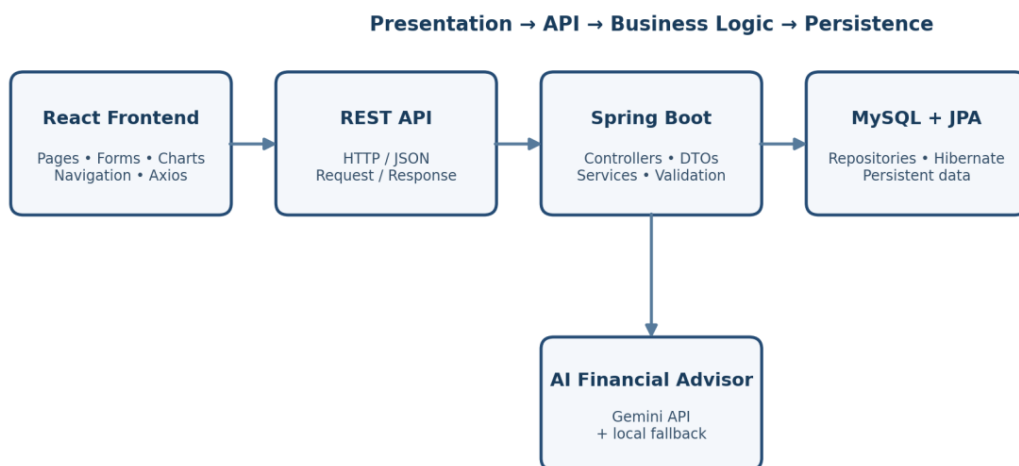
Smart Expense Tracker with AI Financial Advisor is a full-stack personal finance management web application developed to bring income recording, expense tracking, budgeting, financial reporting, analytics and AI-assisted guidance into a single system. The project addresses a practical problem: personal financial information is often distributed across notes, spreadsheets, banking applications and separate tracking tools, making it harder to maintain a clear and timely view of financial activity.

The application combines a React-based user interface with a Java 22 and Spring Boot 3.5.6 backend and a MySQL relational database. Communication between the browser and backend is handled through REST APIs. The backend follows a layered structure involving controllers, data-transfer objects, services, repositories and entities, while the frontend provides dedicated user-facing areas for authentication, dashboard analytics, income, expenses, budgets, reports, AI advice and profile management.

The project also demonstrates how conventional software-engineering functions can be combined with AI assistance. The AI Financial Advisor uses financial context available inside the application, such as spending and budget information, to generate contextual suggestions through the Gemini API. A local fallback recommendation mechanism is available when the external AI service is unavailable or not configured. The AI feature is positioned as a financial-awareness aid rather than regulated investment advice.

From an internship and learning perspective, the project provides practical exposure to full-stack application development, RESTful communication, relational data persistence, CRUD operations, validation, reporting, responsive web interaction and integration with an external AI service. It also demonstrates the relationship between user requirements, system design, implementation, testing and documentation within an SDLC-oriented development process.

Figure 1.1: High-level concept of the application



External AI is accessed by the backend; the fallback keeps advice available when Gemini is unavailable.

Description. The diagram shows the major technical layers used by the application. The React frontend handles presentation and user interaction; REST requests connect it to Spring Boot; the backend applies business rules and communicates with MySQL through JPA/Hibernate. AI requests are coordinated by the backend rather than exposing the Gemini key to the browser.

1. Introduction (Continued)

The application is intended for students, working professionals and general users who need a centralised method for organising personal financial information. The user can register an account, maintain income and expense records, create category-based budgets, monitor actual spending, review reports and obtain contextual financial suggestions. Dashboard summaries and charts make the stored information easier to interpret than raw transaction records alone.

The project includes user-level data protection measures appropriate to its academic scope. Newly registered passwords are protected using BCrypt hashing. Financial operations are scoped to the relevant user, and protected frontend navigation prevents direct access to application pages when a user is not logged in. Input validation and centralised exception handling support predictable application behaviour when invalid or unexpected requests occur.

Reporting is another important part of the application. Users can work with date-oriented financial information and generate reports in PDF and Excel formats. This transforms stored transactions into information that can be reviewed, shared or retained for personal record keeping. Budget monitoring further adds a planning dimension by comparing planned amounts with actual spending and presenting usage, remaining amounts and status information.

The project is currently a functional academic/local full-stack application. Its present scope focuses on the core finance-management workflow and AI-assisted guidance. Production deployment, extensive automated testing, stronger enterprise authentication and authorisation, database migration tooling, monitoring, recurring transactions and more advanced analytics remain areas for future improvement rather than current features.

This report therefore focuses on the purpose, scope, system design, application areas, diagrams, tables, user workflow, practical application, testing status and future scope of the project. The coding/source-code portion is intentionally excluded so that the document remains suitable as an internship project report rather than a source-code manual.

2. Purpose of the Project

The primary purpose of Smart Expense Tracker with AI Financial Advisor is to provide a single web-based environment in which a user can record financial activity, plan budgets, monitor spending, understand financial patterns and obtain AI-assisted guidance. The project converts the common task of maintaining separate financial records into a connected workflow.

2.1 Specific Purposes

- Centralise income and expense records in one application.
- Allow users to organise spending by category and date.
- Provide category-based budget planning for a selected month and year.
- Compare planned budget amounts with actual spending and identify approaching or exceeded limits.
- Present financial summaries through dashboard cards, charts and analytics.
- Generate user-specific PDF and Excel reports with date-oriented filtering.
- Provide contextual AI-assisted suggestions using the financial information available in the application.
- Protect account information and scope financial operations to the relevant user.
- Demonstrate practical full-stack development using modern frontend, backend, database and AI technologies.

The purpose is not to replace banking services, professional financial planners or regulated investment advice. The system is an academic and practical finance-management application intended to improve organisation, awareness and analysis of personal financial information.

3. Scope of the Project

The project scope covers the complete local full-stack application lifecycle from browser interaction through REST communication, backend business logic and database persistence. It also includes reporting, budgeting, analytics, AI-assisted recommendations and user-level data protection.

Table 3.1: Scope coverage of the project

Area	Included in Current Scope
Authentication	Registration, login, logout and protected application navigation
Income	Create, edit, search, delete and date-oriented analysis
Expenses	Create, edit, search, delete, category summaries and analysis
Budgets	Category/month/year budgeting, monitoring, usage and status
Dashboard	Financial summaries, charts, recent activity and budget progress
Reports	User-specific reports, date filtering, PDF and Excel export
AI Advisor	Gemini-based contextual recommendations with local fallback
Profile	Name/email updates and password-change functionality
Notifications	Optional exceeded-budget email notifications
Responsive UI	Desktop and smaller-screen friendly application layout

Description. The table separates the major functional areas that belong to the current project. Production cloud deployment, recurring transactions, advanced analytics and expanded automated testing are deliberately outside the current implementation boundary.

3.1 Scope Boundaries

- Production cloud deployment is not part of the current implementation.
- Extensive automated unit, integration and API testing is a future improvement.
- Advanced enterprise authentication and authorisation controls are future scope.
- Database migration and production monitoring tooling are future scope.
- The AI Advisor provides assistance for financial awareness and is not presented as regulated investment advice.

4. Project Overview and Objectives

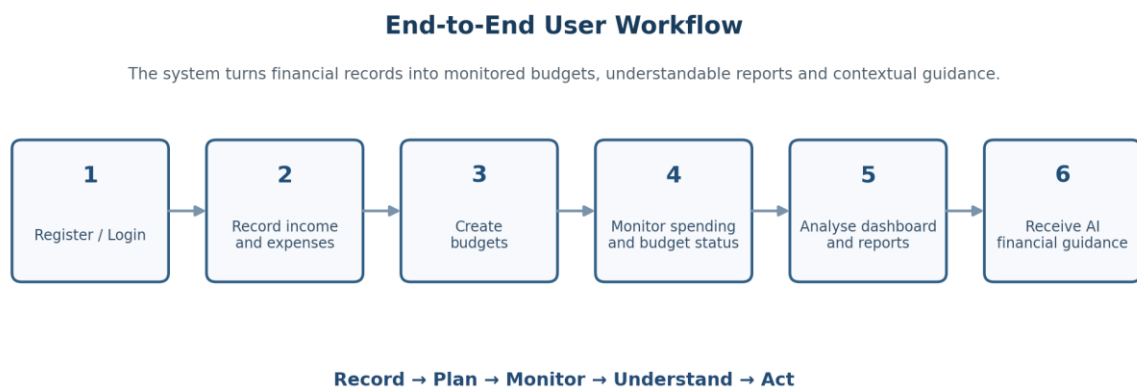
The application is designed around a simple financial-management cycle: record financial activity, plan limits, monitor actual spending, analyse the results and obtain contextual guidance. This approach allows individual modules to contribute to a single user journey instead of operating as disconnected demonstrations.

4.1 Project Objectives

- Provide secure user registration and login with protected navigation.
- Allow users to create, update, search and delete income and expense records.
- Support category-based budgets for selected months and years.
- Calculate budget usage, remaining amount and budget status.
- Provide dashboard summaries, charts and monthly/yearly analytics.
- Generate user-specific PDF and Excel reports.
- Provide Gemini-based AI Financial Advisor functionality with a local fallback.
- Provide profile and account management.
- Persist financial information using MySQL.
- Apply validation, exception handling and ownership checks to support reliable behaviour.

4.2 End-to-End User Journey

Figure 4.1: End-to-end user workflow

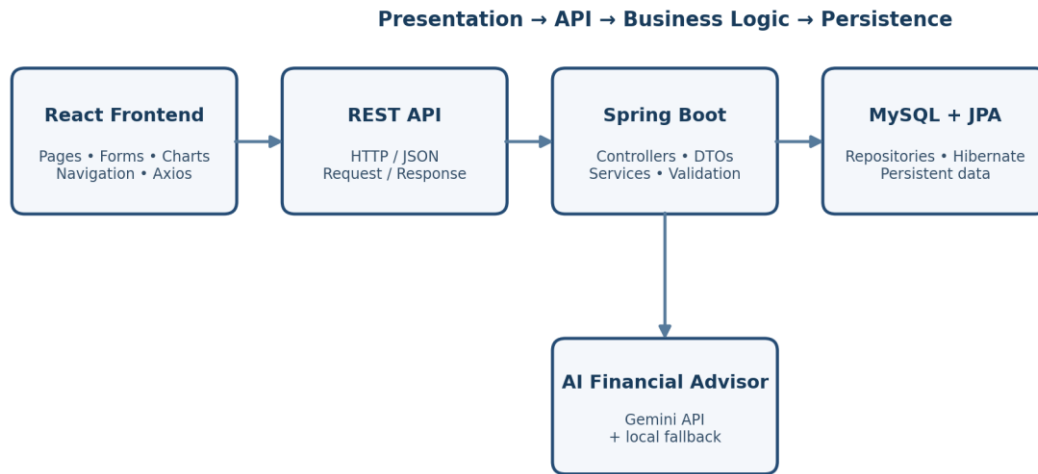


Description. The workflow begins with account access and continues through transaction recording, budgeting, monitoring and analysis before reaching AI-assisted guidance. The sequence illustrates how individual modules combine into one continuous financial-management process.

5. System Architecture

The system uses a layered full-stack architecture. The frontend is responsible for presentation, navigation, forms, charts and API interaction. The Spring Boot backend provides REST endpoints and separates request handling, data transfer, business logic and persistence. MySQL stores user and financial information through Spring Data JPA/Hibernate.

Figure 5.1: High-level system architecture



External AI is accessed by the backend; the fallback keeps advice available when Gemini is unavailable.

Description. The architecture separates concerns so that the user interface does not directly access the database. Requests move through the REST API into backend layers, where validation and business rules are applied before data is persisted or returned.

5.1 Layer Responsibilities

Table 5.1: Major architectural layers

Layer	Responsibility
React Frontend	Pages, forms, navigation, charts, protected UI and API interaction
Controller Layer	Receive HTTP requests, validate boundary inputs and return responses
DTO Layer	Transfer structured request and response data
Service Layer	Apply business rules, ownership checks, budget calculations, reporting and AI orchestration
Repository Layer	Access persistent data through Spring Data JPA/Hibernate
MySQL	Persist users, transactions, budgets, AI-related records and other application data

Description. The table explains why the project is organised into layers. Each layer has a focused responsibility, making the application easier to understand, maintain and extend without mixing presentation, business logic and database concerns.

5. System Architecture (Continued)

5.2 Backend Organisation

The backend is organised into configuration, controller, DTO, entity, repository and service areas. Controllers expose application endpoints; DTOs define structured request/response models; entities represent persisted data; repositories provide database access; and services contain the business rules used by the application.

Table 5.2: Backend project areas

Area	Examples / Responsibility
config	Application and WebClient-related configuration
controller	Authentication, dashboard, income, expense, budget, report, AI and profile endpoints; global exception handling
dto	Login, dashboard, budget progress, summaries, profile, password-change and user data transfer models
entity	User, Expense, Income, Budget and AIAdvice persistence models
repository	Spring Data repositories for application data access
service	Business services for users, expenses, income, budgets, dashboard, reports, AI advice, alerts and email

Description. The table summarises the backend organisation without reproducing source code. It is intended to explain the role of each project area during an internship/project presentation.

5.3 Data Flow

1. A user performs an action in the React interface.
2. The frontend sends a request to the appropriate REST endpoint.
3. The controller receives the request and passes structured data into the service layer.
4. The service layer applies business rules, validation-related checks and user ownership rules.
5. The repository layer communicates with MySQL through JPA/Hibernate when persistence is required.
6. The backend returns a structured response to the frontend for presentation.

6. Application Modules and Functional Areas

The application is divided into functional modules so that each major personal-finance task has a clear user-facing area. Together, these modules provide the complete application workflow.

Table 6.1: Functional module summary

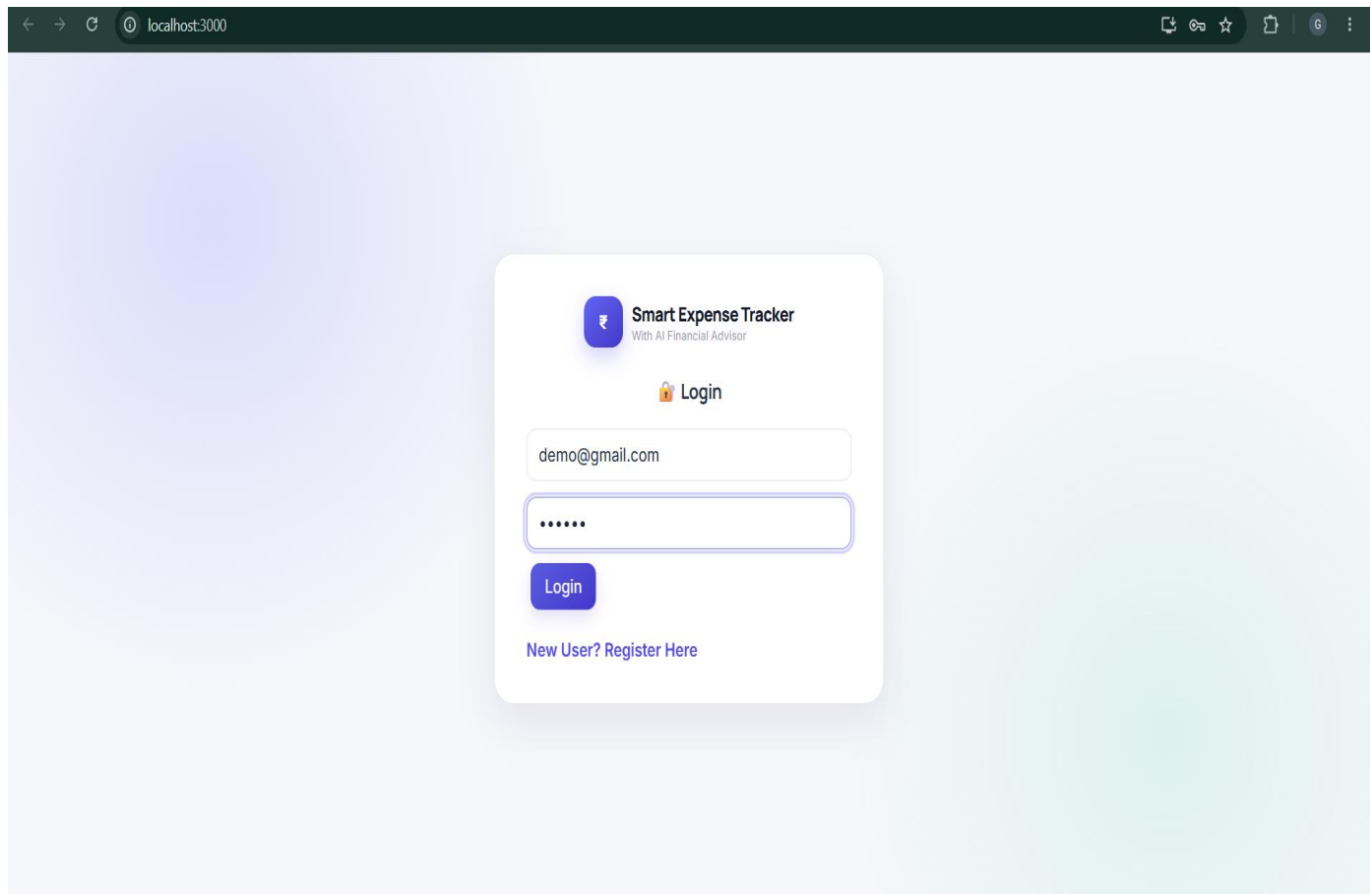
Module	Main Functions
Authentication	Registration, login, logout and protected navigation
Dashboard	Financial overview, summaries, charts, budget progress and recent activity
Income	Income entry management, search and date/month/year analysis
Expenses	Expense CRUD, search, categories and analysis
Budget	Category/month/year budgets, usage, remaining amount and status
Reports	Date-range reporting, PDF export and Excel export
AI Financial Advisor	Contextual financial recommendations and local fallback
Profile	Account information and password management
Notifications	Optional exceeded-budget email notifications

Description. The module table gives a functional view of the application. It intentionally avoids source-code details and instead explains what a user can accomplish through each area.

6.1 Authentication and Profile

The authentication area provides account registration and login. After successful authentication, protected application navigation allows the user to reach the financial modules. Logout clears the local login state and protected navigation prevents unauthorised access to the application interface through normal browser navigation.

Profile management allows a user to update account information and change the account password. Newly registered passwords are protected using BCrypt hashing before storage.



6. Application Modules and Functional Areas (Continued)

6.2 Income Management

The Income module allows the user to maintain records representing money received. The module supports creating, editing, searching and deleting income entries. Date-oriented analysis allows income information to be reviewed over selected periods, including monthly and yearly views where supported by the application.

6.3 Expense Management

The Expense module records money spent by the user. Expense entries can be created, edited, searched and deleted. Categories provide an additional way to group spending so that the dashboard, reports and AI Advisor can work with meaningful financial context.

6.4 Dashboard and Analytics

The Dashboard provides a consolidated view of the financial information maintained by the user. It brings together income, expenses, budgets, recent activity, budget progress and chart-based summaries. This reduces the need to interpret multiple raw transaction lists separately.

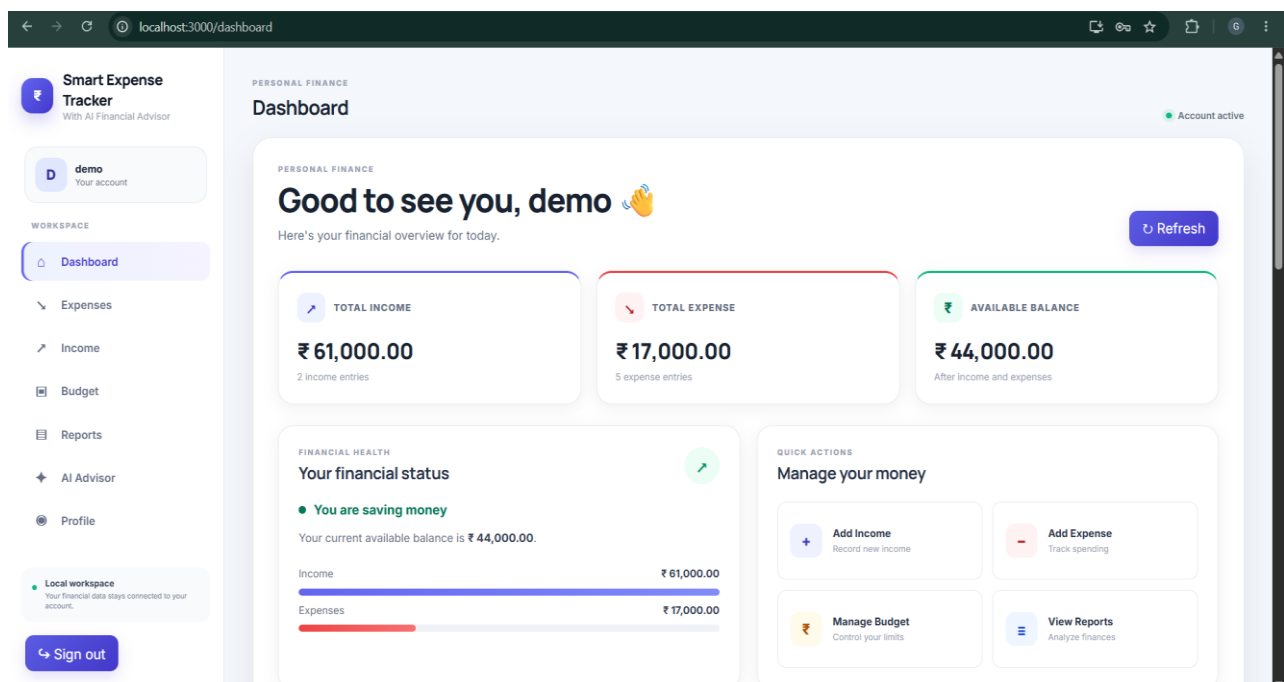
6.5 Responsive User Interface

The frontend is designed as a responsive web interface so that the main application areas remain usable on desktop and smaller screen sizes. Navigation, forms, summaries and reporting areas are presented as user-facing modules rather than requiring technical interaction with the backend.

Table 6.2: Typical information available to the user

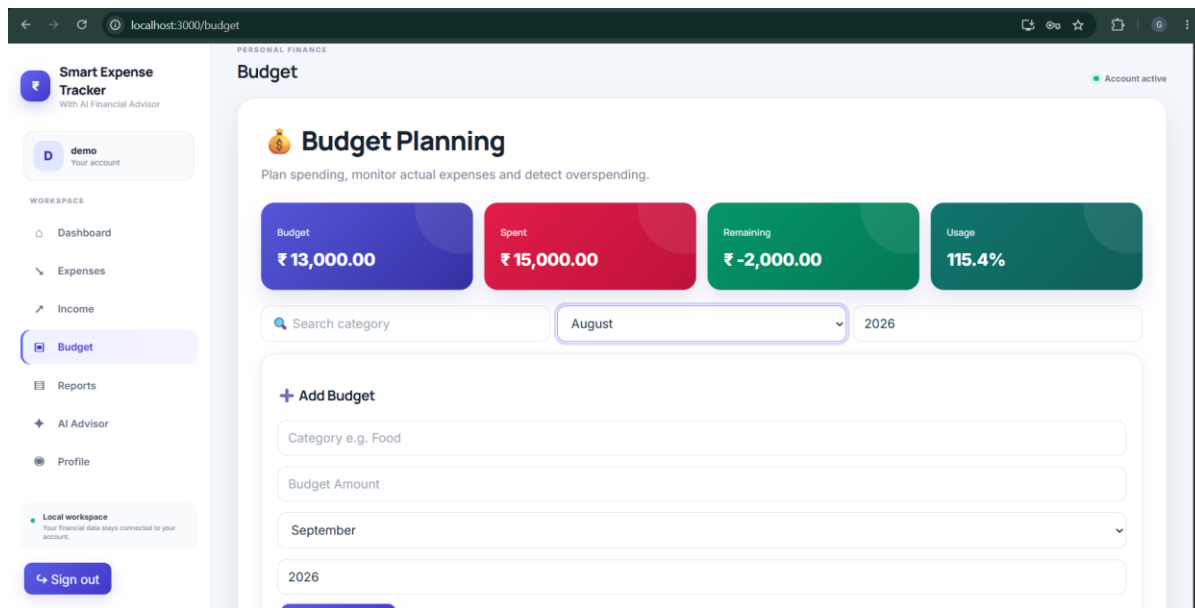
Information	Example Use
Income	Review salary or other recorded income
Expenses	Identify spending by category and date
Budget	Compare planned category limits with actual use
Dashboard	View a consolidated financial summary
Reports	Generate a date-focused financial record
AI Advice	Receive contextual suggestions based on available financial information

Description. The table demonstrates how individual data areas become useful information when connected through the application. The same financial records can support monitoring, reporting and AI-assisted guidance.



7. Budget Monitoring and Financial Analytics

Budget management is a central part of the application because recording expenses alone does not show whether spending is within a planned limit. The Budget module allows category-based planning for a selected month and year and compares the planned amount with actual expenditure.



7.1 Budget Information Presented

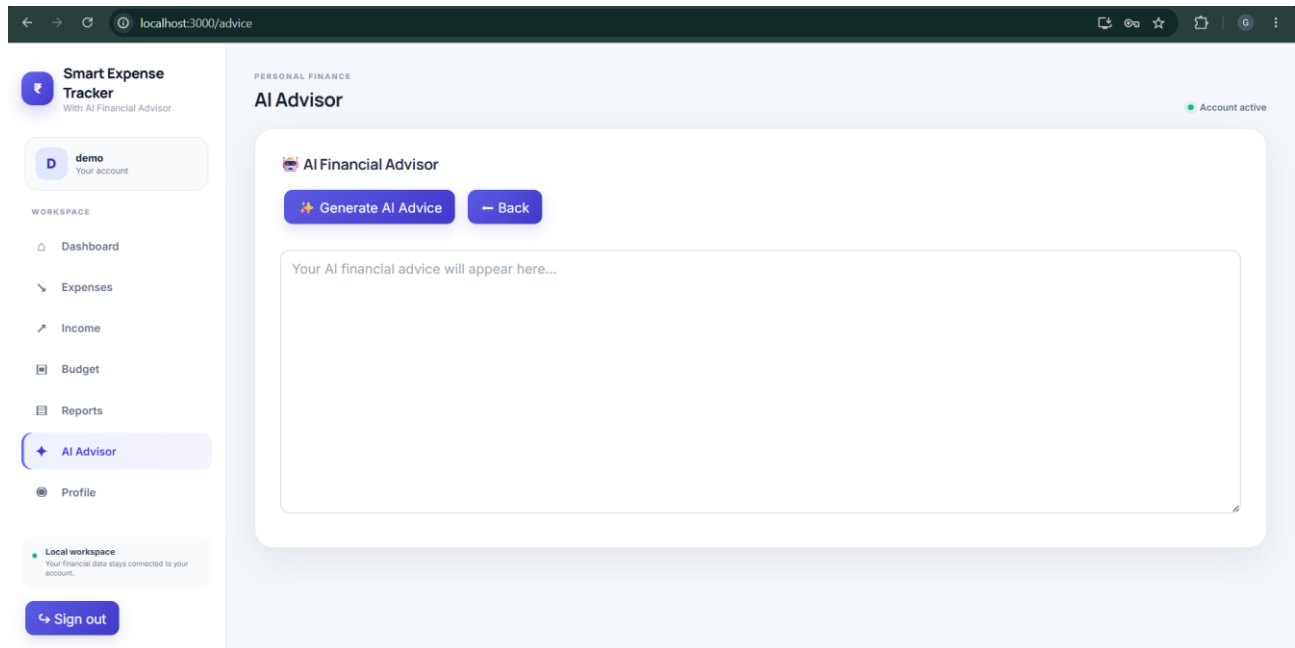
- Budgeted amount for the selected category and period.
- Actual spending recorded for the relevant category and period.
- Remaining amount after accounting for actual spending.
- Percentage of the budget already used.
- Warning state when utilisation reaches 80% or more.
- Over-budget detection when actual spending exceeds the planned limit.

7.2 Dashboard Analytics

The dashboard uses summaries and charts to make financial patterns easier to interpret. Monthly and yearly views help users move beyond individual transactions and observe changes in income, expenses and spending categories over time. The dashboard also brings budget information into the same overview so that financial planning and actual activity can be considered together.

8. Reports and Data Presentation

The reporting area converts stored financial records into user-specific outputs that can be reviewed outside the normal dashboard interface. Reports support date-range filtering and provide PDF and Excel export formats.



8.1 Importance of Reporting

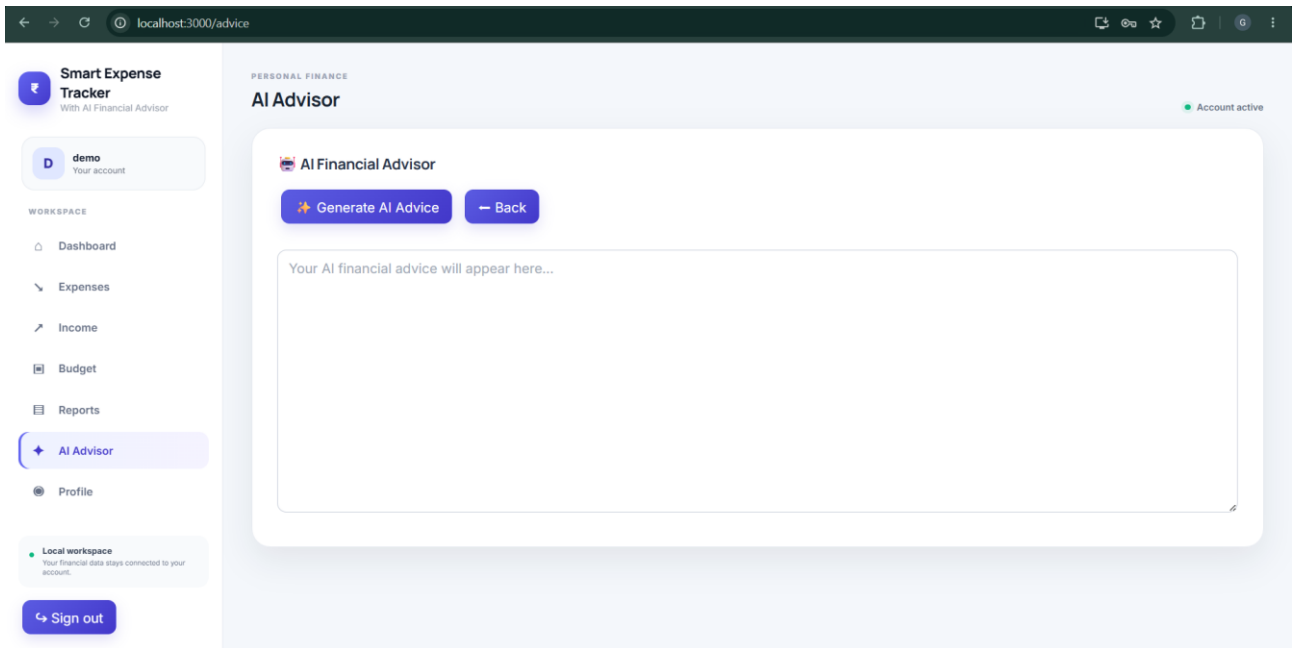
Reporting is useful because financial data becomes more actionable when it can be grouped by period and presented in a structured form. Instead of manually copying transactions into another document, the application can prepare user-specific reports directly from the maintained financial information.

8.2 Data Presentation

The application combines tables, summary values and charts to present information at different levels of detail. Transaction-oriented views support detailed record management, while dashboard and report views support higher-level interpretation.

9. AI Financial Advisor

The AI Financial Advisor is the project component that connects conventional personal-finance management with AI-assisted interpretation. It uses financial context already available inside the application, such as expenses and budgets, to support contextual suggestions.



9.1 Practical Role of AI

- Provide spending suggestions based on available expense information.
- Consider budget context when producing recommendations.
- Help users interpret financial activity in more understandable terms.
- Provide an additional layer of guidance without replacing the underlying financial records or reports.

9.2 Responsible Use

AI output depends on the quality and completeness of the financial context provided to it. For this reason, the feature is presented as an assistance mechanism for financial awareness rather than as professional investment advice. The project keeps the Gemini API configuration on the backend so that the API key is not exposed through the React frontend.

10. Technology Stack and Development Environment

The project uses a conventional full-stack web architecture with React on the client side, Java and Spring Boot on the server side, and MySQL for relational persistence. Additional libraries support charts, reporting, AI communication, email notifications and user interaction.

Table 10.1: Technology stack used in the project

Area	Technology / Tool	Purpose
Frontend	React 19	User interface and component-based web application
Routing	React Router	Client-side navigation and protected routes
HTTP	Axios	Communication with backend REST APIs
Charts	Chart.js / react-chartjs-2	Dashboard visualisation and analytics
UI feedback	SweetAlert2	User-facing success, warning and error messages
Backend	Java 22 + Spring Boot 3.5.6	REST backend and application services
Persistence	Spring Data JPA / Hibernate	Relational data access and object mapping
Database	MySQL 8.x	Persistent storage for users and financial data
Reporting	Apache POI + PDFBox	Excel and PDF report generation
AI	Gemini API + WebClient	AI-assisted financial recommendations
Email	Spring Boot Mail	Optional budget-related email notifications
Build	Maven Wrapper	Backend dependency management and build
Development	Eclipse + Visual Studio Code	Backend and frontend development

Description. *The technology table provides a project-level view of the tools and their roles. It avoids source-code listings while giving an evaluator a clear understanding of how the application is assembled.*

10.1 Development Environment

The backend was developed and validated using Eclipse IDE for Enterprise Java and Web Developers with Java 22 and Maven-based project management. The frontend is developed using Visual Studio Code and follows the Create React App workflow, using `npm start` for local development.

11. Security, Validation and Data Protection

The project includes several measures intended to protect account and financial information within its academic/local scope. These measures focus on password protection, user ownership, protected navigation, validation and configuration safety.

Table 11.1: Current security and reliability measures

Measure	Current Application Behaviour
BCrypt hashing	Newly registered passwords are hashed before storage
Protected navigation	Application pages require a logged-in user state
Ownership checks	Financial operations are scoped to the relevant user
Input validation	Invalid input is checked at appropriate application boundaries
Exception handling	Centralised handling improves consistency of error responses
Secret configuration	Database, Gemini and mail credentials are intended to be supplied through environment variables

Description. The table distinguishes current implemented safeguards from future production security work. The application uses BCrypt password hashing; it does not describe password storage as reversible encryption.

11.1 Configuration Safety

Sensitive values such as database passwords, Gemini API keys and SMTP credentials should not be embedded in source files committed to version control. Environment variables are used to keep these values outside the application source configuration.

11.2 Data Ownership

The backend applies ownership checks when working with user-specific financial records. This is important because a financial application must ensure that one account cannot intentionally operate on another account's income, expense or budget data through the same API workflow.

12. Application of the Project

The application can be used as a central personal-finance workspace. Its practical value comes from connecting record keeping, planning, monitoring, reporting and guidance rather than treating each task as a separate utility.

12.1 Practical Use Cases

- A student can record regular income and expenses and observe category-level spending.
- A working professional can create monthly budgets and compare actual spending with planned limits.
- A user can generate a date-range report for personal financial record keeping.
- A user can inspect dashboard charts to understand spending patterns.
- A user can request AI-assisted suggestions using the financial context already maintained in the system.
- A user can manage account details and change the account password through the Profile module.

12.2 Example User Scenario

A typical demonstration can begin with a new user registering and logging in. The user can then record salary income of ₹50,000, add Food expenses of ₹4,000 and Transport expenses of ₹2,000, and create a Food budget of ₹10,000 for the selected month. The dashboard and budget area can then be used to review spending and budget utilisation, followed by report generation and the AI Financial Advisor.

Table 12.1: Example demonstration data

Type	Category	Example Amount	Purpose
Income	Salary	₹50,000	Show available income
Expense	Food	₹4,000	Demonstrate expense entry
Expense	Transport	₹2,000	Demonstrate multiple categories
Budget	Food	₹10,000	Demonstrate budget monitoring
Budget	Transport	₹5,000	Demonstrate category planning

Description. *The example data provides a simple, repeatable demonstration sequence. The values are illustrative and are intended to show how the modules interact rather than represent actual financial advice.*

13. Testing and Current Project Status

The current application has been manually validated through the main local development workflows. The testing approach focused on confirming practical user behaviour and integration between the React frontend, Spring Boot backend and MySQL database rather than claiming exhaustive automated coverage.

Table 13.1: Validation areas

Validation Area	Current Status
Backend startup	Validated with Java 22 and Spring Boot 3.5.6
Database connectivity	Validated with MySQL in the local environment
Registration and login	Core authentication workflow manually tested
Dashboard	Dashboard access and summary presentation tested
Income and expense operations	Create/delete and related workflows tested
Budget operations	Creation, monitoring, progress values and status tested
Reports	Reporting workflow and export functionality tested
AI Advisor	AI Advisor workflow manually tested
Profile	Profile navigation and account-management workflow tested
Logout / protected navigation	Logout and protected navigation behaviour tested
Build	Backend Maven package build successfully validated

Description. The validation table records the areas that were checked during the project review. Automated unit, integration and API test coverage is not presented as complete and remains an area for future improvement.

13.1 Current Technical Status

The project has reached a working local full-stack state. The backend starts in Eclipse with Java 22 and Spring Boot 3.5.6, connects to MySQL and exposes the required REST endpoints. The React frontend runs through the Create React App workflow using `npm start`. The demonstrated application covers authentication, dashboard access, income and expense management, budgeting, reporting, AI advice, profile navigation, logout and protected-route behaviour.

14. Limitations and Future Scope

The current implementation is suitable as an academic/local prototype, but several improvements would be required before treating it as a production financial platform. These limitations are not failures of the academic objective; they identify practical areas for future engineering work.

14.1 Current Limitations

- The application is currently intended for local academic use rather than cloud production deployment.
- Automated unit, integration and API test coverage can be expanded.
- Production-grade authentication and authorisation controls can be strengthened.
- Analytics can be extended with additional trend and comparison views.
- AI recommendations can be improved with richer context and more explainable outputs.

14.2 Future Scope

- Recurring income and recurring expense support.
- Advanced spending trends and category comparisons.
- Improved AI context handling and explanation of recommendations.
- Additional export formats and report customisation.
- Database migration tooling for controlled schema evolution.
- Production monitoring, logging and operational observability.
- Secure hosted secret management.
- Production/cloud deployment of the frontend, backend and database.

15. Conclusion

Smart Expense Tracker with AI Financial Advisor demonstrates how a practical personal-finance problem can be addressed through a complete full-stack web application. The project brings together income and expense management, category-based budgeting, dashboard analytics, reporting, profile management and AI-assisted financial guidance within one connected workflow.

The React frontend provides the user-facing experience, while the Java and Spring Boot backend separates API handling, business logic and persistence. MySQL provides structured relational storage, and additional components support PDF/Excel reporting, optional email notifications and Gemini-based AI recommendations with a local fallback. These technologies demonstrate practical integration rather than isolated technology exercises.

The project also demonstrates important software-engineering practices such as layered architecture, REST API communication, validation, user-ownership checks, password hashing and configuration safety. Manual validation has confirmed the principal application workflows in the local development environment, making the system suitable for academic demonstration and further refinement.

Overall, the project provides a useful internship learning outcome because it covers the complete journey from identifying a real-world requirement to designing, integrating, testing and documenting a working software system. The future scope provides a clear path towards stronger testing, richer analytics, improved AI context, production security and deployment.

16. References

1. React Documentation, Meta Open Source. <https://react.dev/>
2. Spring Boot Reference Documentation, Spring. <https://docs.spring.io/spring-boot/>
3. Spring Data JPA Documentation, Spring. <https://spring.io/projects/spring-data-jpa>
4. MySQL 8.0 Reference Manual, Oracle. <https://dev.mysql.com/doc/refman/8.0/en/>
5. Apache Maven Documentation. <https://maven.apache.org/guides/>
6. Apache POI Documentation, Apache Software Foundation. <https://poi.apache.org/>
7. Apache PDFBox Documentation, Apache Software Foundation. <https://pdfbox.apache.org/>
8. Google Gemini API Documentation, Google AI for Developers. <https://ai.google.dev/gemini-api/docs>
9. Axios Documentation. <https://axios-http.com/docs/intro>
10. Chart.js Documentation. <https://www.chartjs.org/docs/latest/>
11. Eclipse IDE Documentation and Downloads. <https://www.eclipse.org/ide/>

These references were used as the technical documentation basis for the frameworks, libraries and services described in the project report. Access dates may be added by the student or institution if required by the internship-report format.

17. Bibliography

1. Sommerville, Ian. Software Engineering. Pearson.
2. Pressman, Roger S. and Maxim, Bruce R. Software Engineering: A Practitioner's Approach. McGraw-Hill.
3. Oracle. Java Platform Documentation and API Reference.
4. Spring. Spring Framework and Spring Boot technical documentation.
5. Meta Open Source. React documentation and learning resources.
6. Oracle. MySQL documentation and reference materials.
7. Apache Software Foundation. Maven, POI and PDFBox project documentation.
8. Google AI for Developers. Gemini API documentation and integration guidance.
9. Project documentation prepared for Smart Expense Tracker with AI Financial Advisor, including the project synopsis, SDLC report and setup documentation.